

Inverse Aerodynamic Design of a Converging-Diverging Nozzle

A 1D Inviscid Shape Optimization Study via Genetic Algorithms

Danial Moghadam

Aerodynamics Graduate — dan998m@gmail.com

August 25, 2026

Abstract

This report details the development of an automated inverse-design computational pipeline for optimizing the internal wall geometry of a converging-diverging nozzle. Utilizing a 1D, incompressible, inviscid flow solver based on the Karimian & Schneider block tri-diagonal method, a Genetic Algorithm (GA) is deployed to manipulate a 7th-degree Bézier curve boundary. The optimization objective is to achieve a targeted pressure distribution designed to mimic boundary layer protection mechanisms via a Stratford curve approximation. The custom solver is first validated against exact analytical formulations to ensure physical accuracy, the genetic algorithm then utilizes geometry parametrizations to converge on physically valid, mathematically optimized aerodynamic shape subject to a 2.0-meter packaging constraint.

1 Introduction and Engineering Objective

In the development of modern aerodynamics, Optimization has become an important tool in modern aerodynamic design. Modern optimization techniques primarily rely on two different methodologies; Forward vs. Inverse Design. Forward design is the process of evaluating the performance and characteristics of a predetermined geometry. Forward design is computationally expensive, since a great number of iterations are required to land on a final design. Inverse design, on the other hand, works backwards. In inverse design process, the engineer dictates a theoretically efficient flow dynamics and computational algorithm works towards a designing a geometry that satisfies the optimal criteria.

The goal of this study is to develop an inverse design tool that is capable of designing a geometry for a converging-diverging nozzle that follows a strict pressure curve designed to satisfy a pressure-recovery criterion associated with boundary-layer separation avoidance. The physical design space is constrained to a total length of $L = 2.0$ m, with a geometric throat fixed at 30% of the axial length ($c_{th} = 0.3$). The optimization algorithm must uphold conservation of mass while preventing the generation of non-physical geometries that would otherwise have poor aerodynamic performance.

2 Numerical Methodology

2.1 Governing Equations

The flow is modeled as steady, one-dimensional, incompressible, and inviscid. Under these assumptions, the Navier-Stokes equations reduce to the continuity and Euler momentum equations:

$$\frac{d(uA)}{dx} = 0 \quad (\text{Continuity}) \quad (1)$$

$$u \frac{du}{dx} = -\frac{1}{\rho} \frac{dP}{dx} \quad (\text{Momentum}) \quad (2)$$

To ease solver implementation, density is factored out of the numerical integration, allowing the solver to compute purely in terms of kinematic pressure ($P_{kin} = P/\rho$).

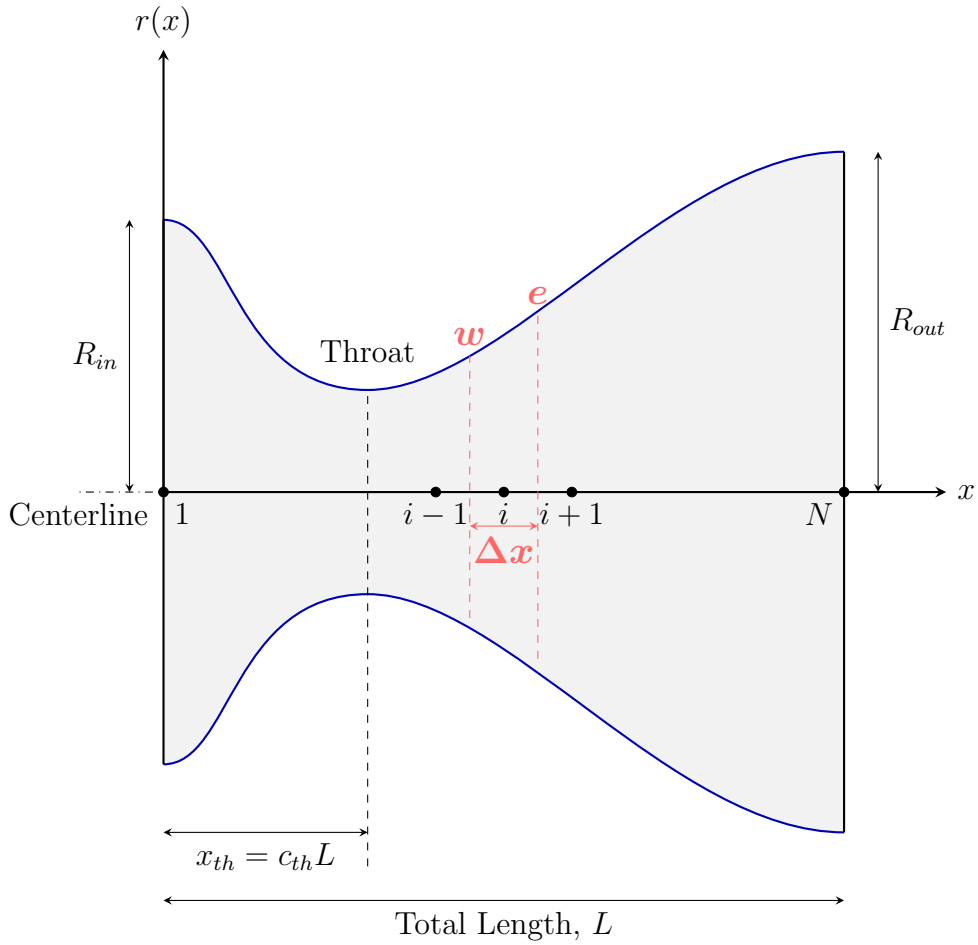


Figure 1: Geometric parameterization and discretization of the converging-diverging nozzle. The domain is divided into N nodes, bounded by the inlet (R_{in}) and exhaust (R_{out}), with the discrete control volume for node i shown with its east (e) and west (w) faces.

2.2 Variable Area Formulation (Karimian & Schneider)

The domain is discretized into $N = 63$ nodal control volumes (Figure 1). A fundamental challenge in numerical modeling of Navier-Stokes arises from the convective acceleration term ($u \frac{du}{dx}$), which is non-linear. Furthermore, a simple colocated

grid often results in a non-physical zig-zag pressure field famously known as the "checkerboard problem".

The Karimian & Schneider method prevents this decoupling by distinguishing between the convecting mass-conserving velocity ($\hat{u}$) and the convected momentum velocity (u).

Table 1: Nomenclature and spatial discretization conventions.

Grid Convention	Description
Uppercase (U, P, A)	Values evaluated at discrete nodal centers
Lowercase (u, p)	Interpolated values evaluated at control volume faces
Variable	Physical Quantity
P, p	Pressure (static or kinematic)
U, u	Convected momentum velocity
$\hat{u}$	Mass-conserving (advecting) velocity
A	Cross-sectional area
Subscript	Spatial Location
P	Primary evaluation node
E, W	Adjacent East and West nodes
e, w	East and West faces of the control volume
in, out	Global inlet and outlet boundaries
th	Geometric throat

2.3 Interior Nodes Discretization

For all internal nodes, the Conservation of Mass dictates that the mass flux entering the control volume equals the mass flux exiting:

$$\rho \hat{u}_e A_e - \rho \hat{u}_w A_w = 0 \quad (3)$$

To prevent non-physical oscillations, the mass-conserving velocity at the east face ($\hat{u}_e$) is mathematically formulated to account for both the pressure gradient and the geometric area gradient:

$$\hat{u}_e = \frac{U_P + U_E}{2} - \frac{\Delta x}{2\rho \hat{u}_e A_e} \left[A_e \left(\frac{P_E - P_P}{\Delta x} \right) - \rho \hat{u}_e^{-2} \left(\frac{A_E - A_P}{\Delta x} \right) \right] \quad (4)$$

A similar formulation is applied to the west face ($\hat{u}_w$), anchoring the continuity equation to the physical boundaries of the control volume.

The Momentum Equation balances the convective fluxes against the pressure forces acting on the projected nodal area A_P :

$$(\rho \hat{u}_e A_e) u_e - (\rho \hat{u}_w A_w) u_w + A_P (p_e - p_w) = 0 \quad (5)$$

where the convected velocities are derived using local pressure gradients, evaluated using values from the previous iteration ($\bar{u}$):

$$u_e = U_P - \frac{1}{2\rho \hat{u}_e} [P_E - P_P] \quad \& \quad u_w = U_W - \frac{1}{2\rho \hat{u}_w} [P_P - P_W] \quad (6)$$

2.4 Boundary Condition Formulations

The matrix of coefficients enforces conservation laws at either far end: (Inlet & Outlet)

Inlet (Node 1): The mass conservation for first node is calculated as follows:

$$\rho A_e \hat{u}_e = \rho A_1 U_{inlet} \quad (7)$$

While the momentum equation strictly sets $U_1 = U_{inlet}$.

Outlet (Node N): The downstream boundary relies on the upstream mass flux to define the exit velocity U_N :

$$\rho A_w \hat{u}_w = \rho A_N U_N \quad (8)$$

Simultaneously, the momentum equation locks the final node to the target outlet pressure, $P_N = P_{outlet}$.

3 Custom Flow Solver Architecture

A significant portion of this study was devoted to the the development of the custom solver (for educational purposes), rather than relying on commercial solvers. The solver is fully implicit, coupling the continuity and momentum equations into a $2N \times 2N$ block tri-diagonal matrix ($[A][x] = [B]$). The solution vector x sequentially packs pressure and velocity: $x = [P_1, U_1, \dots, P_N, U_N]^T$. Every odd row models Continuity, and every even row models Momentum.

3.1 Algebraic Expansion of Internal Nodes

By substituting the velocity definitions back into the conservation equations and isolating the variables, the coefficients matrix was constructed as follows;

```
1 for k = 3:(N-2)
2     i = 2*k - 1; % Odd Row: Continuity Equation
3
4     Mat_A(i,i-2) = -(Area(k)+Area(k-1))/uhat(k-1); %
5     % Coeff for P_{k-1}
6     Mat_A(i,i-1) = -(Area(k)+Area(k-1)); %
7     % Coeff for U_{k-1}
8     Mat_A(i,i) = ((Area(k-1)+Area(k))/uhat(k-1)) + ((Area(k)
9     +Area(k+1))/uhat(k)); % Coeff for P_k
10    Mat_A(i,i+1) = Area(k+1)-Area(k-1); %
11    % Coeff for U_k
12
13    B(i) = 2*uhat(k-1)*(Area(k)-Area(k-1)) - 2*uhat(k)*(Area(k
14    +1)-Area(k));
```

Listing 1: Internal Node Matrix Assembly (MATLAB)

3.2 The Iterative Advection Update

The matrix coefficients rely on the guessed mass-conserving velocity field, `uhat`. After solving the linear system, the advection field is updated:

```

1 for j=1:N-1
2     new_uhat(j) = 0.5*(U(j)+U(j+1)) - 0.5*((P(j+1)-P(j))/uhat(
        j)) + uhat(j)*(Area(j+1)-Area(j))/(Area(j)+Area(j+1));
3 end
4 uhat = new_uhat;

```

Listing 2: Mass-Conserving Velocity Update

This loop runs until the residual error drops below the 0.1% tolerance threshold.

4 Analytical Baseline Validation

Before delving into the optimization process, the custom solver was subjected to testing in order to verify its validity against analytical solutions. And since our solver is inviscid and one-dimensional, analytical solution exists at every point using Conservation of Mass and Bernoulli's equation. Hence, a classical piecewise parabolic converging-diverging nozzle was selected and generated to guarantee a zero-slope throat ($\frac{dr}{dx} = 0$), preventing any geometric singularities from artificially tripping the solver. Moreover, the solver is generalized to solve for all sorts of nozzles given certain geometric parameters. (And is available for anyone to use) To show the mathematical stability and physical accuracy of our custom solver, the numerical output was plotted against the exact analytical solutions [Figure 2].

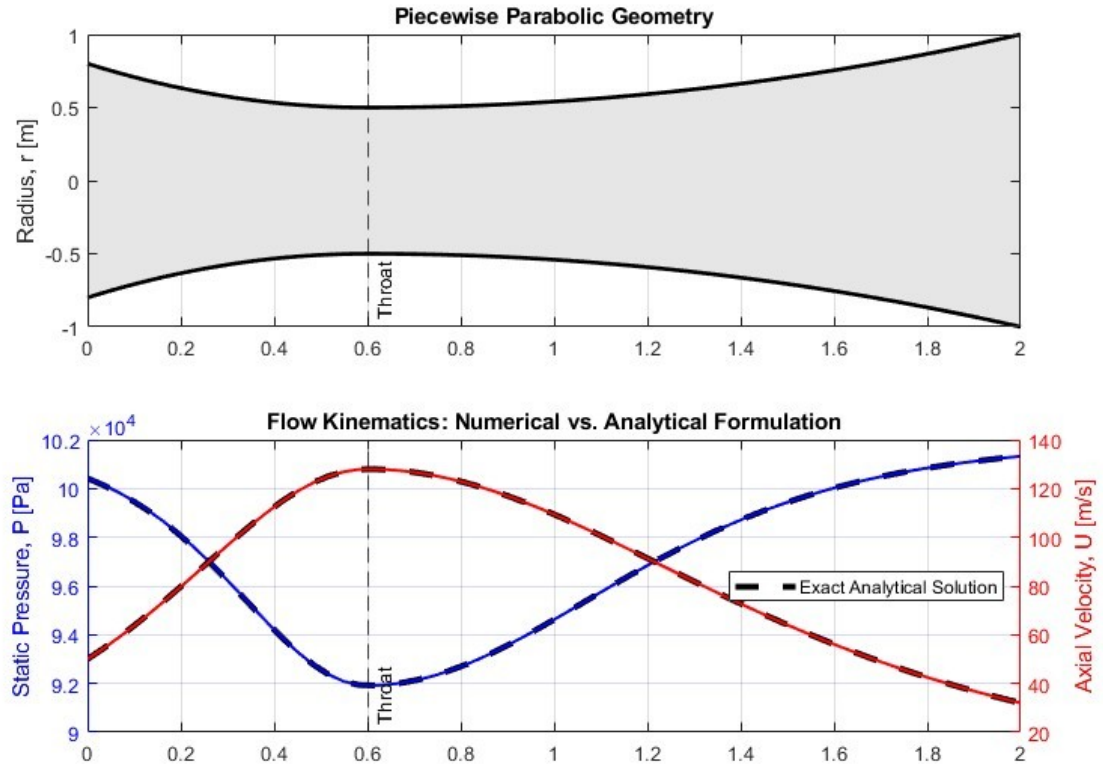


Figure 2: Baseline solver validation. The top panel illustrates the piecewise parabolic geometry. The bottom panel demonstrates the excellent agreement that exists between the custom solver (solid blue and red lines) and exact analytical solution (dashed black lines).

5 Target Pressure Synthesis and Physics Anchoring

With the solver validated, the inverse-design framework requires a mathematical target. To ensure aerodynamic viability, the target pressure distribution (P_{target}) must be analytically synthesized to respect viscous boundary layer survival mechanisms, and globally anchored to obey energy conservation.

5.1 Analytical Inlet Anchoring

The nozzle outlet is open to the atmosphere ($P_{out} = 101325$ Pa). The arbitrary packaging limits fix the radii at $R_{in} = 0.8$ m and $R_{out} = 1.0$ m, with an inlet velocity of $U_{in} = 50$ m/s. Bernoulli's Equation is applied to find the inlet static pressure:

$$(\rho = 1.225 \text{ kg/m}^3)$$

$$P_{in} = P_{out} + \frac{1}{2}\rho \left[\left(U_{in} \frac{R_{in}^2}{R_{out}^2} \right)^2 - U_{in}^2 \right] = 100420.95 \text{ Pa} \quad (9)$$

Anchoring the upstream boundary mathematically prevents the optimizer from clashing with the solver's inherent energy conservation.

5.2 Piecewise Aerodynamic Profile

The Converging Section (Favorable Gradient): A half-period cosine wave is utilized because its derivative is exactly zero at the peaks and troughs, preventing localized acceleration spikes that prematurely thicken the boundary layer.

$$P_{target}(x) = P_{th} + (P_{in} - P_{th}) \left[\frac{1}{2} \left(1 + \cos \left(\pi \frac{x}{x_{th}} \right) \right) \right] \quad (10)$$

The Diverging Section (Stratford Approximation): A Stratford approximation is used here since it can provide maximum recovery of pressure in the shortest possible distance without separating the boundary layer and the exponential decay approximation is computationally cheap to work with during the optimization loop. As a result, it allows the flow out of the throat to have a steep pressure gradient because the boundary layer here is thin and the flow energetic. It is then followed by a very gentle pressure recovery towards the outlet. It's a mathematical tool that allows for the flow to ride along the absolute limit of the wall shear stress $\tau_w = 0$ without separating.

$$P_{target}(x) = P_{out} - (P_{out} - P_{th}) e^{-k(x-x_{th})} \quad (11)$$

6 Optimization Architecture

6.1 Geometric Parameterization and Parameter Independence

The nozzle wall is parameterized using a 7th-degree Bézier curve driven by Bernstein polynomials since this curve provides excellent geometric control and flexibility.

$$r(\xi) = \sum_{i=0}^n \binom{n}{i} (1-\xi)^{n-i} \xi^i R_i \quad (12)$$

where $\xi = x/L$ is the normalized spatial coordinate, and R_i represents the discrete control radii.

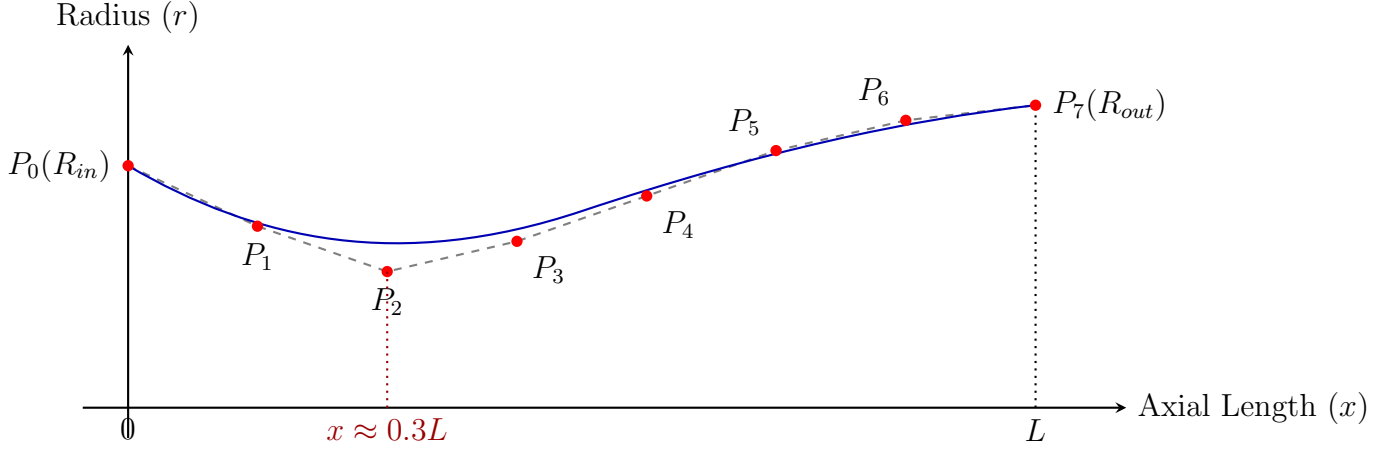


Figure 3: Parametric representation of the nozzle wall using an 8-point Bézier curve. The algorithm holds the boundary points fixed and only vertically manipulates the intermediate control points.

And to prevent aerodynamic overfitting, a parameter independence study was carried out to determine the sweet spot number of intermediate control points.

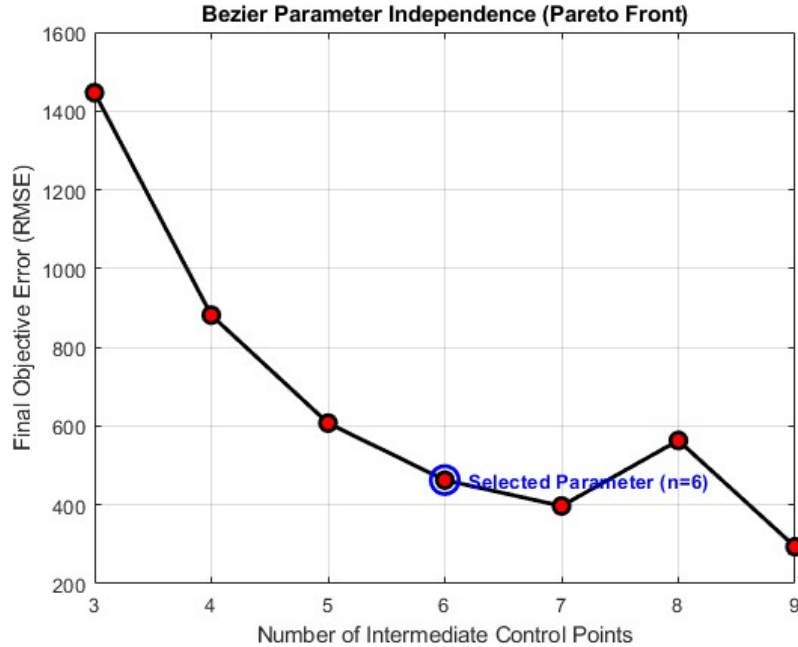


Figure 4: Parameter independence study (Pareto front) for the Bézier curve control points.

As can be observed in (Figure 4), 6 intermediate control points were selected. The selected design is the compromise between pressure-matching performance and geometric regularity; lower degree polynomials lacked the geometric flexibility, while

higher degree polynomials (e.g., $n = 8$) triggered the objective function to generate non-physical wall oscillations and caused the objective function's geometric roughness penalty.

6.2 Objective Function and Geometric Regularization

The Genetic Algorithm (GA) evaluates the fitness of guessed geometries by calculating the Root Mean Square Error (RMSE) between the solver's output and the target curve.

To prevent the algorithm from mathematically "cheating" by generating squiggly, non-physical walls that average out to the correct pressure, a geometric roughness penalty was introduced based on the second derivative of the control points.

```

1 function J = evaluate_nozzle_objective(R_opt, x, L, N, U_inlet
  , P_out_kin, P_targ_kin, R_in, R_out)
2   R_full = [R_in, R_opt, R_out];
3   [~, A, ~] = generate_bezier_geometry(R_full, L, N);
4
5   try
6     [P_kin, ~, ~] = solve_inviscid_nozzle(x, A, U_inlet,
7       P_out_kin);
8     RMSE = sqrt(mean((P_kin - P_targ_kin).^2));
9
10    % Geometric Roughness Penalty (Minimizes High-
11    % Frequency Wall Waviness)
12    roughness = sum(diff(R_full, 2).^2);
13    J = RMSE + (1000 * roughness); % Lambda weight = 1000
14  catch
15    J = 1e9; % Divergence Penalty
16  end
17 end

```

Listing 3: Objective Function with Roughness Penalty

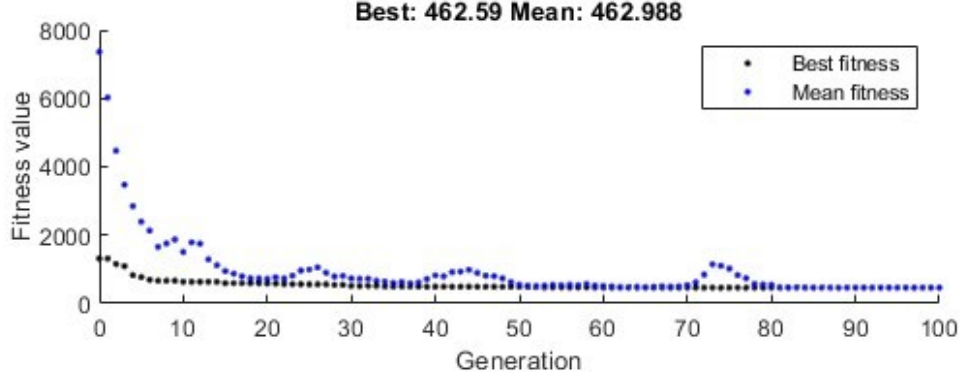


Figure 5: Evolution of the genetic algorithm over N generations.

The optimization loop was run for 100 generations. As can be seen, the algorithm actively explored new designs by deviating from best fitness value. The spike between 70 and 80 generations is a clear example of this phenomenon. These deviations before final convergence verify the fact that the algorithm was not stuck in some local minimum. By the final iterations, the population mean settles with the best fitness score of ($J \approx 462.6$) indicating the narrowing down of the search and finalizing the optimum geometry.

7 Results and Engineering Analysis

The Genetic Algorithm was initialized with a set of points with an emphasis on placing the throat at the ($x = 0.3L$). These control points were selected on the basis of having an aerodynamically blind baseline.

As illustrated in Figure 6, the baseline geometry (gray dashed lines) failed to meet our design requirements by a considerable amount. The large discrepancy particularly around the throat location indicates a mismatch with our localized pressure requirements. By controlling the Bézier points, the optimized geometry constricted the throat causing a large pressure drop, this can be safely done since our flow is energetic and the boundary layer is relatively thin. Past the throat, the algorithm tries to recover pressure by closely hugging our target Stratford approximation which ensures quick pressure recovery riding the limits of what the boundary layer can withstand. The asymptotic behavior of the optimized pressure curve towards the outlet of the nozzle is precisely to prevent disruptions in the flow.

The resulting optimized pressure field (solid blue line) shows excellent agreement with the theoretical Stratford recovery approximation curve. The slight rounding of the pressure minimum at the throat ($x = 0.6$ m) is a direct consequence of the geometric roughness penalty. Instead of generating a sharp cusp at the throat, the algorithm generated a smooth compromise that protects the downstream boundary layer envelope.

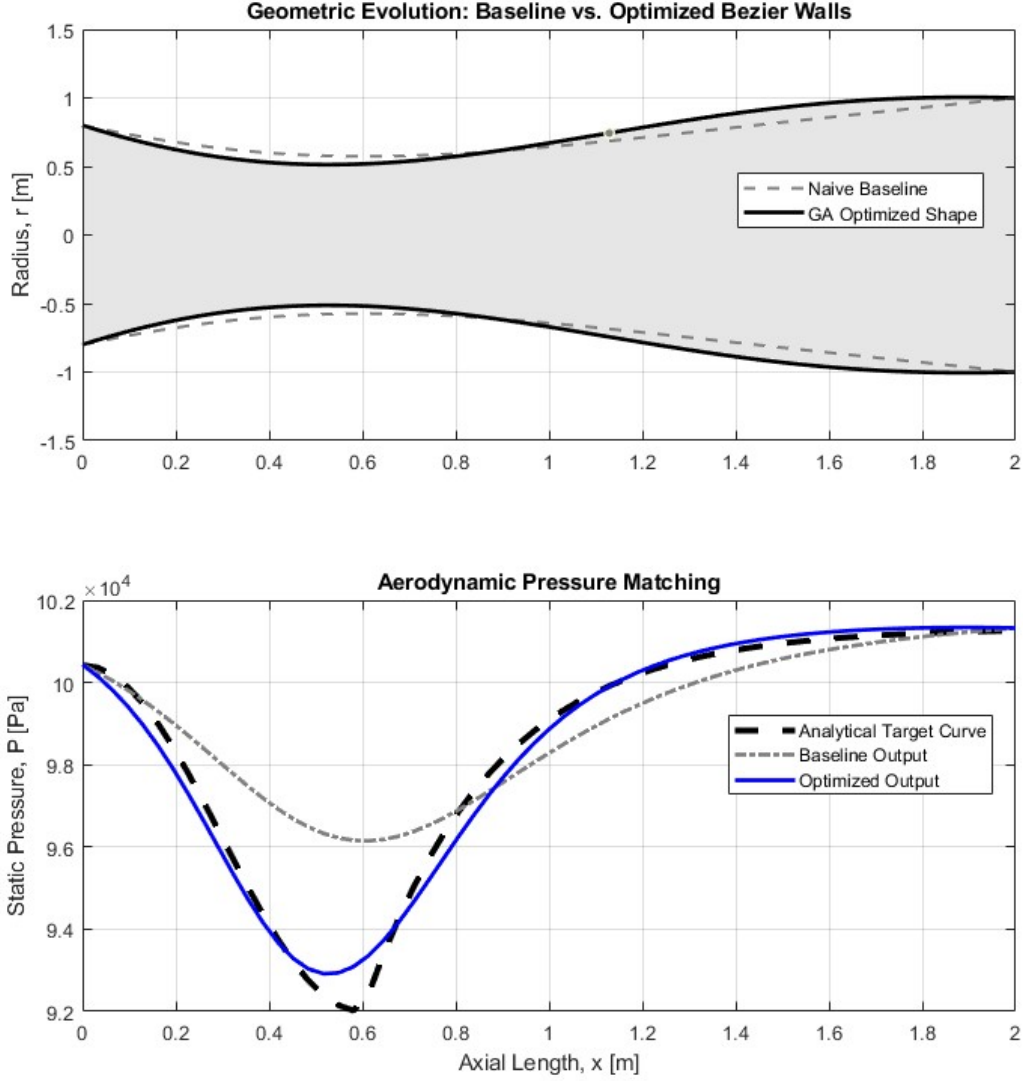


Figure 6: Evolution of the 1D nozzle geometry. The optimizer successfully manipulated the intermediate Bézier control points, shifting from the aerodynamically naive baseline (dashed grey) to a mathematically smoothed shape that tracks hugs the Stratford pressure recovery target (solid blue).

7.1 Justification of the Inviscid Framework for Viscous Optimization

While real-world aerodynamic flows are almost always viscous, the pressure field within the flow is largely determined by the outer inviscid region. According to classical boundary layer theory developed by Prandtl, the viscous layer is directly governed by this external pressure gradient ($\frac{dP}{dx}$). Therefore, if we opted to use the full viscous Navier-Stokes equations to tackle this problem, it would not only be computationally expensive but also largely unnecessary during the initial stages of design exploration. By using a fast 1D inviscid solver instead, the Genetic Algorithm can rapidly evaluate thousands of design iteration in a matter of seconds. And to further bridge the gap between real-world flows and inviscid math, viscous con-

straints were embedded into the pressure target curve (i.e. the objective function) through the use of the Stratford approximation. So in essence, the inviscid solver is actively trying to match the target curve which itself has the viscous considerations baked in. As a result, the optimizer lands on a final design that promotes boundary layer separation avoidance. despite the solver itself being inviscid.

8 Conclusion

This study investigates the coupling of classical numerical fluid dynamics with modern optimization frameworks. By mathematically constraining the search space for our converging-diverging nozzle via a Bézier curve, and pre-calculating the physical boundary states to satisfy energy conservation laws, the Genetic Algorithm was guided to produce a refined, aerodynamic nozzle geometry. Effectively, transitioning from a ground-up 1D matrix solver into an automated inverse-design tool.

References

- [1] Darbandi, M., *A Momentum Variable Calculation Procedure for Solving Flow at all Speeds*, PhD Thesis, Department of Mechanical Engineering, University of Waterloo, Waterloo, Ontario, Canada, 1996.
- [2] Karimian, S. M. H., and Schneider, G. E., “Pressure-Based Control-Volume Finite Element Method for Flow at All Speeds Using Momentum Variables,” *AIAA Journal*, or related foundational works developed at the University of Waterloo CFD Group, 1995–1996.
- [3] Patankar, S. V., *Numerical Heat Transfer and Fluid Flow*, Hemisphere Publishing Corporation, 1980.
- [4] Bézier, P., *Numerical Control: Mathematics and Applications*, John Wiley & Sons, 1972.